

SE4050 – Deep Learning

Week 5 – Convolutional Neural Networks (Part 2)

Dr. Mahima Weerasinghe, *MIET*

Content

- From last lecture
 - Exercise 1
- Loss functions of CNNs
- Full example explanation
- Transfer learning
 - Exercise 2
- Data augmentation
 - Exercise 3

From last lecture

- Introduction to CNNs

 - Convolution Operation

 - Filters/Kernels

 - Feature Maps

- Pooling Layers

 - Max Pooling

 - Average Pooling

 - Min Pooling

 - Global Pooling

- Padding and Stride

- Fully Connected Layers

- Architectures of CNNs

 - LeNet-5

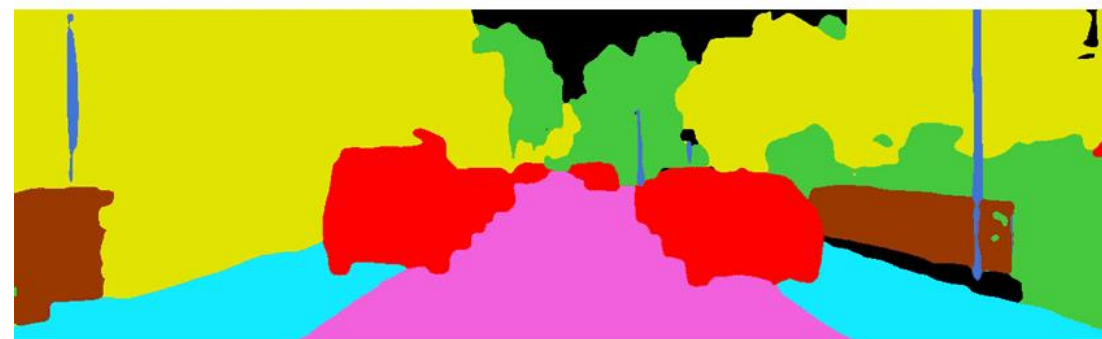
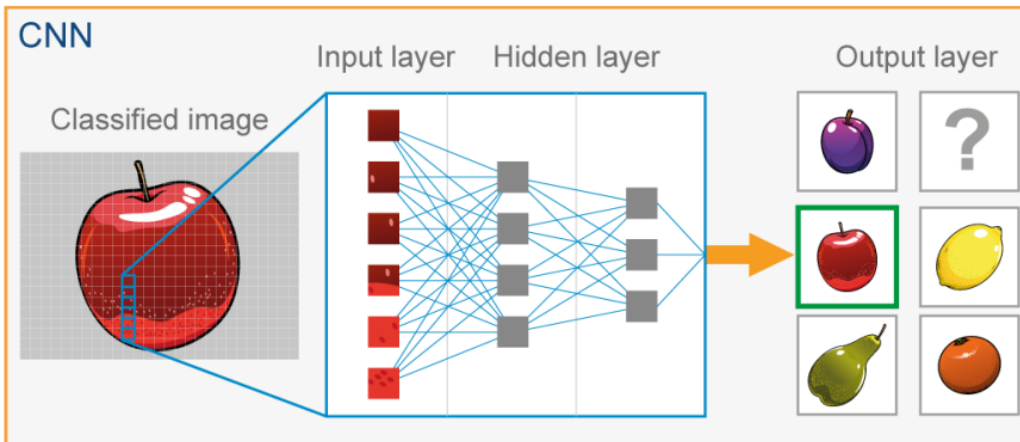
 - AlexNet

 - VGGNet

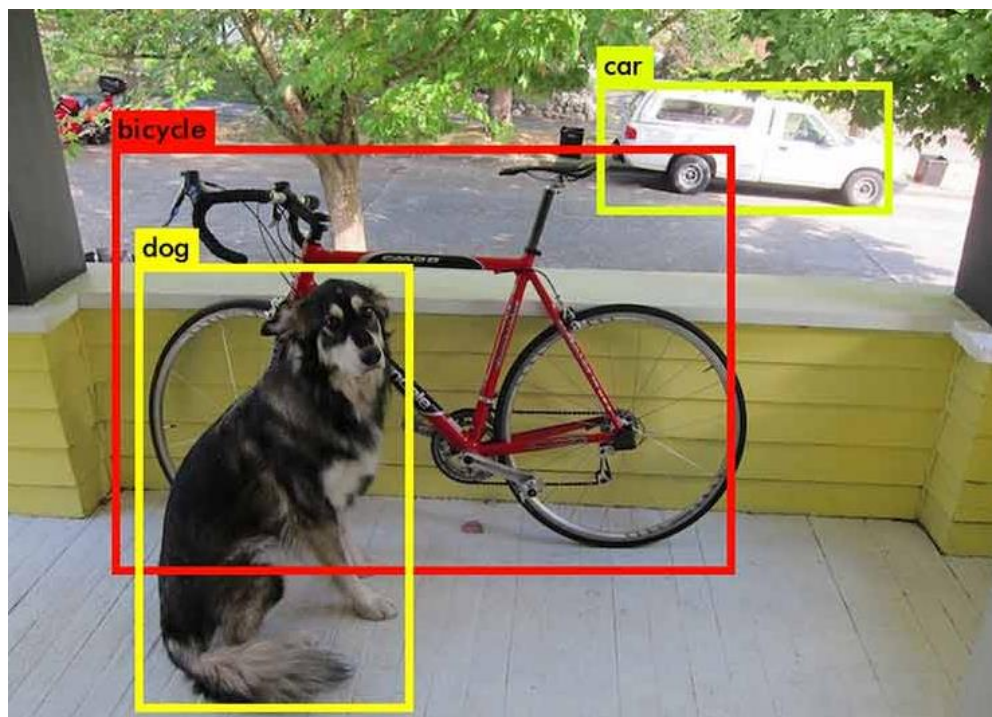
 - ResNet

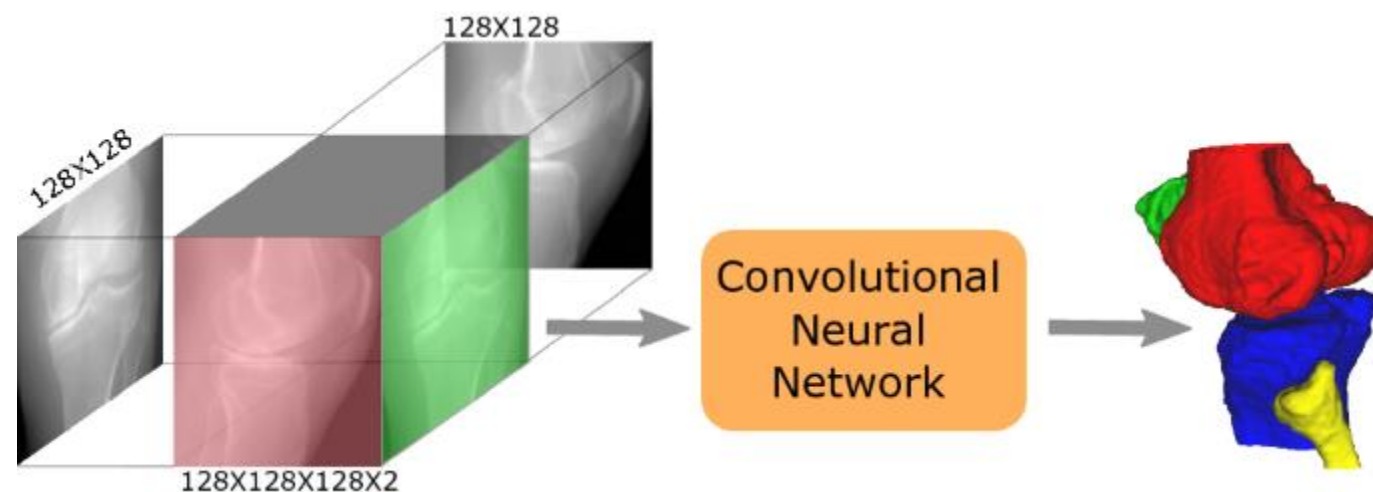
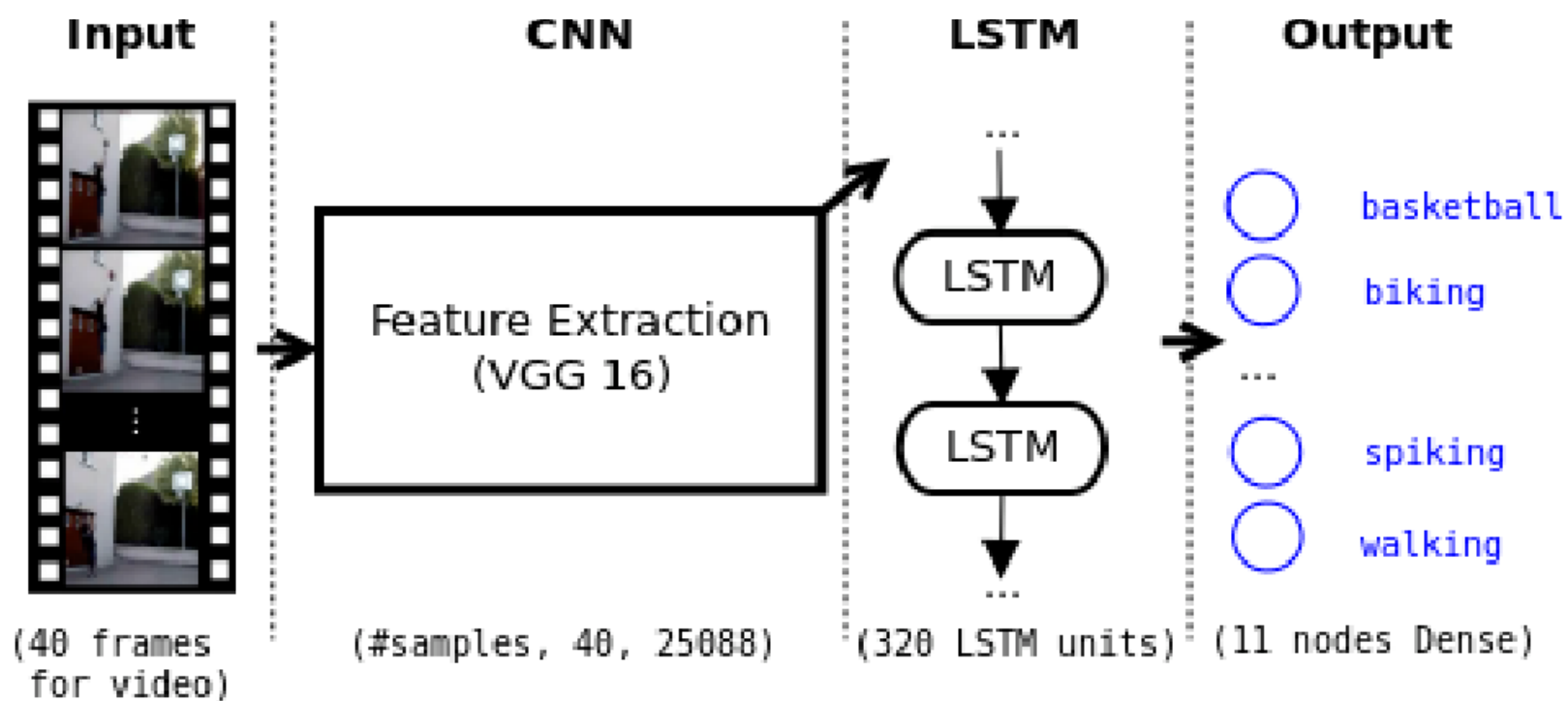
Exercise 1

1. Imagine an input with 32x32 pixels. If the image is grayscale, how many data points are required to represent the image ?
2. If a 5x5 kernel is used, calculate the size of the output feature map. (Stride = 1 and no padding)
3. How many trainable parameters are there for each kernel? (Consider a bias per kernel)
4. If 10 such kernels are there, calculate the output size.
5. How many connections are there in total at the convolutional layer?
6. Consider a max pooling layer of 2x2 with stride 2, what is the size of the resulting feature map?
7. If a dense layer of 200 neurons are used, how many connections are present in this layer? (consider one bias value per feature map)



Road	Sidewalk	Building	Fence
Pole	Vegetation	Vehicle	Unlabel





Loss functions

Cross-Entropy Loss (Categorical Cross-Entropy)

Usage: Primarily used in multi-class classification tasks.

$$L = - \sum_{i=1}^C y_i \log(p_i)$$

Explanation: Cross-Entropy Loss measures the dissimilarity between the true label distribution and the predicted probability distribution. It penalizes the model more when it assigns low probabilities to the correct class. This loss function is especially effective when softmax activation is used in the output layer of a CNN.

Applications: Object detection, image classification, and natural language processing tasks.

Binary Cross Entropy

Usage: Used for binary classification tasks where each example belongs to one of two classes.

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Explanation: This is a specific case of cross-entropy loss applied to binary classification. The loss function penalizes the model when it is confident in the wrong prediction.

Applications: Binary classification tasks

Dice Loss

Usage: Commonly used in image segmentation tasks

$$L = 1 - \frac{2 \cdot |X \cap Y|}{|X| + |Y|}$$

where X and Y are the predicted and actual sets of pixels.

Explanation: Dice Loss is based on the Dice coefficient, a measure of overlap between two samples. It is particularly useful for tasks where the goal is to maximize overlap between predicted and actual classes, like segmenting images.

Applications: Image segmentation tasks in medical imaging or autonomous driving

Focal Loss

Usage: An extension of cross-entropy loss, used for handling class imbalance in tasks like object detection.

$$L = -\alpha(1 - p_t)^\gamma \log(p_t)$$

where p_t is the probability of the correct class, α is a weighting factor, and γ is a focusing parameter.

Explanation: Focal Loss reduces the relative loss for well-classified examples and focuses on harder, misclassified examples. It is especially useful in scenarios where some classes are much more frequent than others.

Applications: Object detection tasks like in the RetinaNet model

Sparse Categorical Cross-Entropy Loss

Usage: Similar to categorical cross-entropy but used when labels are provided as integers instead of one-hot encoded vectors.

$$L = -\log(p_{y_i})$$

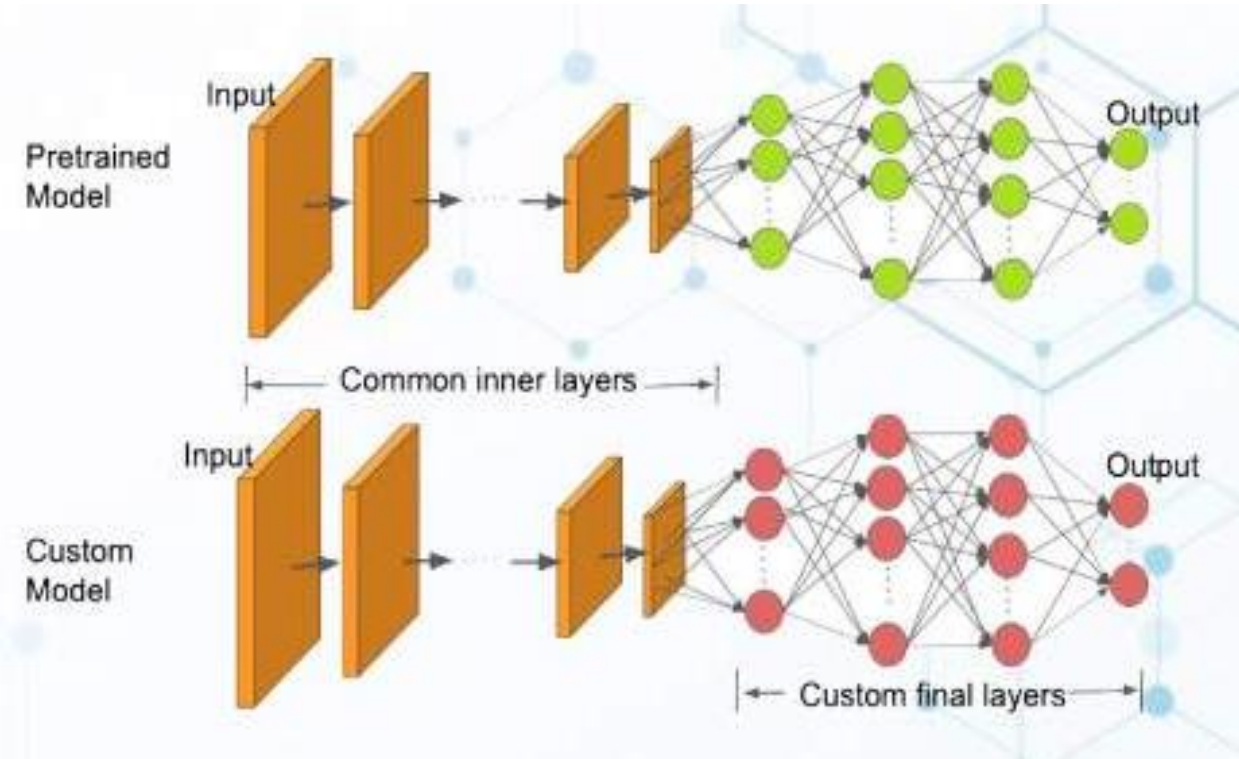
where p_{y_i} is the predicted probability of the correct class.

Explanation: This loss function is computationally more efficient than categorical cross-entropy when dealing with a large number of classes, as it bypasses the need to one-hot encode labels.

Applications: Multi-class classification tasks where classes are represented as integers

Transfer Learning

Transfer learning as a general term refers to reusing the knowledge learned from one task for another. Specifically for convolutional neural networks (CNNs), many image features are common to a variety of datasets (e.g. lines, edges are seen in almost every image). It is for this reason that, especially for large structures, CNNs are very rarely trained completely from scratch as large datasets and heavy computational resources are hard to come by.



Steps in transfer learning

Select a Pre-trained Model – Choose a pre-trained CNN model that suits the task and dataset. Popular choices include VGG, ResNet, Inception, or MobileNet. These models are typically available in deep learning libraries like TensorFlow or PyTorch.

Load Pre-trained Model – Load the pre-trained CNN model without the top (fully-connected) layers. This allows us to leverage the pre-trained model's learned features.

Customize the Model – Add new layers on top of the pre-trained model to adapt it to your specific task. These layers should include a suitable architecture for your task, such as fully-connected layers, dropout layers, or convolutional layers. Adjust the number of neurons or classes based on your specific requirements.

Freeze Pre-trained Layers – Freeze the weights of the pre-trained layers to prevent them from being updated during training. This ensures that the pre-trained features are retained and not modified.

Prepare Data – Preprocess your dataset according to the input requirements of the pre-trained model. This may involve resizing, normalizing, or augmenting the images.

Train the Model – Train the model using your dataset. Only the newly added layers on top of the pre-trained model will be trained while the pre-trained layers remain frozen.

Fine-tuning (Optional) – If you have sufficient data and want to further improve performance, you can unfreeze some of the pre-trained layers and fine-tune them along with the new layers. This allows the model to adapt to the specific features of your dataset.

Evaluate and Test – Evaluate the trained model using validation data or cross-validation techniques. Measure metrics such as accuracy, loss, precision, or recall to assess performance. Finally, test the model on unseen data to get an estimate of its real-world performance.

Exercise 2

<https://colab.research.google.com/drive/1FCErS39f9H04uLXLgP65Y8KMUIb5H-Ck?usp=sharing>

1. Explain the code using comments.
2. Use ResNet pertained layers and compare the results

Image augmentation



Original



Horizontal Flip



Vertical Flip



Horizontal + Vertical



Color Profile 1



Color Profile 2



Color Profile 3



Color Profile 4



Rotate Left



Rotate Right



Noise 1



Noise 2



Crop 1



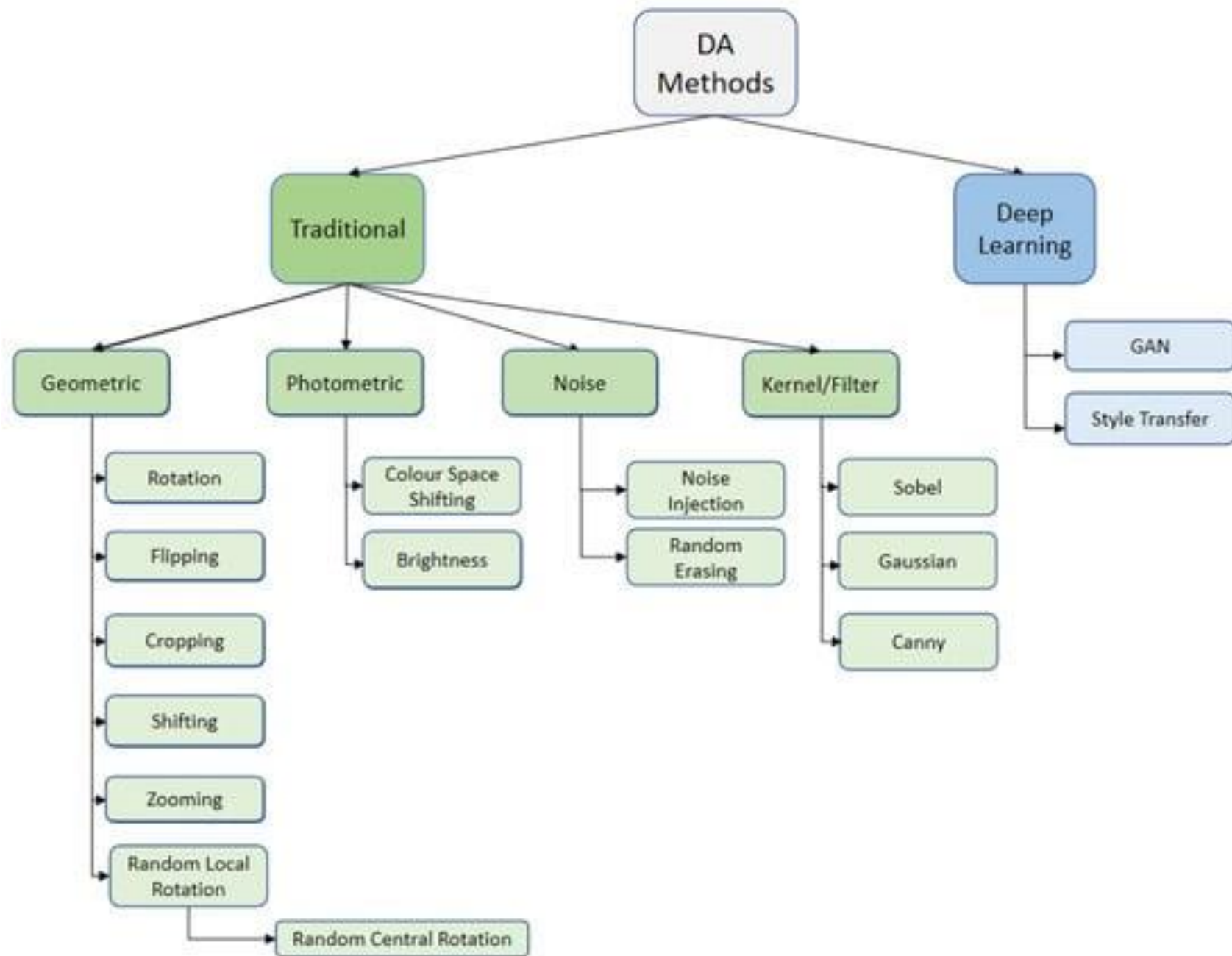
Crop 2



Resize 1



Resize 2



Exercise 3

What is the use of Data Augmentation in the perspective of CNNs? Discuss in detail.

Resources

More on Loss functions

<https://www.kaggle.com/code/wolfmedal/loss-function-in-cnn>

Augmentation Script

<https://www.kaggle.com/code/eugeniyosetrov/using-data-augmentation-for-cnn>

Stanford CNN content – 2024

<https://cs231n.github.io/>